

Sk Miraj
Department of Computer Science
and Engineering (IoT, CS, BT)
University of Engineering
and Management Kolkata, India
sk.miraj2023@iem.edu.in

Richak Ganguly
Department of Computer Science
and Engineering (IoT, CS, BT)
University of Engineering
and Management Kolkata, India
richak.ganguly2023@iem.edu.in

Soumajit Ghosh
Department of Computer Science
and Engineering (IoT, CS, BT)
University of Engineering
and Management Kolkata, India
soumajit.ghosh2023@iem.edu.in

Abstract—The Internet of Vehicles (IoV) has become an important part of modern intelligent transportation systems by enabling communication between vehicles, roadside infrastructure, and cloud platforms. However, the highly dynamic nature of vehicular networks and continuously changing traffic conditions make efficient task processing difficult. Vehicles generate a huge amount of real-time data, but their onboard computational resources are limited. Because of this, task offloading has become essential for reducing delay and improving overall system performance.

This paper proposes a mobility-aware hybrid ACO–DRL task offloading algorithm where ACO performs initial resource exploration and PPO-based DRL continuously refines decisions using network state feedback. The proposed model uses a mobility-aware edge–cloud architecture where computational tasks are intelligently distributed among vehicles, edge servers, and cloud platforms based on network conditions, resource availability, and task requirements. ACO helps in making fast initial offloading decisions, while DRL continuously learns from the environment to improve adaptability and optimization over time.

The proposed framework was evaluated through simulation under different traffic and mobility conditions. Experimental results show that the hybrid ACO–DRL approach reduces communication delay, improves task completion rate, and achieves better resource utilization compared to conventional task offloading methods. The study demonstrates that combining optimization-based and learning-based approaches can provide a more adaptive, scalable, and efficient solution for real-time IoV applications.

Index Terms—Task Offloading in Internet of Vehicles Using Hybrid ACO–DRL.

I. INTRODUCTION

A. Background of IoV

The Internet of Vehicles (IoV) is an advanced extension of the Internet of Things (IoT) that enables intelligent communication among vehicles, roadside infrastructure, edge servers, and cloud platforms. By integrating wireless communication, sensing technologies, and distributed computing resources, IoV facilitates real-time information exchange and supports various applications such as autonomous driving, traffic monitoring, route optimization, accident prevention, and smart transportation management.

With the rapid growth of connected vehicles and intelligent transportation systems, large volumes of data are continuously generated by onboard sensors and communication modules. Efficient processing and management of this data have become critical requirements for ensuring reliable and low-latency vehicular services.

B. Importance of Task Offloading

Modern vehicles execute numerous computationally intensive tasks, including object detection, traffic analysis, nav-

igation assistance, and driver safety monitoring. However, onboard vehicle resources are often limited in terms of processing power, memory, and energy availability. Performing all computations locally may result in increased execution delay and reduced system efficiency.

Task offloading provides an effective solution by transferring computational tasks from vehicles to nearby edge servers or cloud infrastructures. Through intelligent task allocation, offloading can significantly reduce latency, improve resource utilization, and enhance the overall Quality of Service (QoS) of vehicular applications.

C. Challenges in IoV

Despite its advantages, task offloading in IoV environments faces several challenges. The highly dynamic nature of vehicular networks leads to frequent topology changes, unstable communication links, and varying network conditions. Vehicle mobility further complicates decision-making because resource availability continuously changes over time.

Additionally, edge servers may become overloaded during peak traffic conditions, while cloud-based processing introduces communication delays due to longer transmission distances. Therefore, developing intelligent and adaptive task offloading strategies remains a significant research challenge in IoV systems.

D. Motivation

The increasing demand for real-time vehicular applications requires efficient mechanisms for reducing latency and improving computational performance. Traditional cloud-centric solutions often fail to meet the stringent delay requirements of safety-critical applications. Although edge computing provides faster processing capabilities, efficient resource allocation remains a challenge in dynamic traffic environments.

Recent developments in AI, particularly deep reinforcement learning (DRL), have shown great potential in solving sequential decision-making tasks in dynamic edge computing, exhibiting high proficiency in practice [20], [21], [24]. Motivated by these developments, this research explores the integration of Ant Colony Optimization (ACO) and Deep Reinforcement Learning (DRL) to develop an intelligent task offloading framework capable of adapting to changing network conditions and vehicle mobility patterns.

E. Research Contributions

The major contributions of this research are summarized as follows:

- A comprehensive study of task offloading challenges in Internet of Vehicles environments.
- A comparative analysis of Ant Colony Optimization (ACO) and Deep Reinforcement Learning (DRL) based task offloading techniques.
- The development of a Hybrid ACO–DRL framework for intelligent task allocation and resource management.
- Mathematical modeling of task completion time, communication delay, and makespan optimization.
- Performance evaluation under different traffic conditions to analyze latency, task success rate, and system stability.

F. Paper Organization

The remainder of this paper is organized as follows. Section II presents the literature review and discusses existing task offloading approaches in IoV environments. Section III describes the research gap and problem statement. Section IV explains the proposed Hybrid ACO–DRL methodology and system architecture. Section V presents the implementation details and mathematical model. Section VI discusses the experimental setup, results, and performance analysis. Finally, Section VII concludes the paper and outlines future research directions.

II. LITERATURE SURVEY

The offloading of tasks in the Internet of Vehicles (IoV) represents a growing research topic due to the need for low latency, quick computation, and reliable communication in today's vehicular applications. Cloud computing is unable to meet the real-time requirements of vehicular applications due to increased communication latencies and congestions in the network. Therefore, researchers have explored methods including edge computing, optimization techniques, artificial intelligence, and hybrid computing to enable task offloading in IoV.

Hou et al. have proposed an architecture based on roadside units (RSUs) for vehicular edge computing. This design focused on minimizing latency by executing computational tasks using nearby edge servers instead of relying on distant cloud servers. The proposed scheme improved the efficiency of communication and reduced the delay in executing tasks. The scheme exhibited poor performance in case of dynamic mobility due to increased difficulty in maintaining constant communication with RSUs.

Zhang et al. presented a framework based on Multi-access Edge Computing (MEC) for intelligent task scheduling in vehicular environments. The scheme was characterized by the dynamic allocation of bandwidth and computational resources depending on the urgency and priority of tasks. According to experimental analysis, the proposed scheme exhibited improvements in terms of energy efficiency and task processing speed. However, the complexity of the optimization problem increased considerably with the increase in size of the vehicular network.

Deep learning and reinforcement learning methods have been used extensively in intelligent task offloading schemes.

Liu et al. have proposed a Deep Reinforcement Learning (DRL) based task scheduling scheme that is adaptable to changes in network status and traffic conditions. The proposed scheme exhibited better adaptability and lesser processing delay compared to other heuristics. However, the proposed scheme required extensive training periods, large datasets, and huge computational resources, making its deployment in real-time difficult.

Some researchers have explored hybrid computing techniques where the computation load is shared between edge and cloud servers. For instance, Alghamdi and Liao proposed a hybrid computing scheme whereby delay-sensitive tasks are executed at the edge while large computational tasks are offloaded to the cloud. The approach offered advantages in terms of scalability and edge server overload mitigation. However, communication overhead was increased due to coordination between edge and cloud layers. Task offloading among vehicles has gained momentum in recent times. Zhang and Chen designed a peer-to-peer offloading approach whereby nearby vehicles use each other's idle computing capacity to improve system performance. This technique reduces dependence on static infrastructure and maximizes resource utilization in densely populated areas. Security and trust management issues are, however, important in such a scenario since not all vehicles within the vicinity can be treated as reliable computing nodes.

Various optimization algorithms and game theory models have been used to intelligently allocate tasks in IoV. Tang et al. developed an offloading approach based on game theory where each vehicle independently determines the best way to offload its tasks using the available network and system costs. This method improved load balancing and resource distribution. In addition, ACO-based strategies achieved fast convergence and low computational complexity, thus making them applicable in light vehicular systems. These techniques, however, lacked adaptive learning ability and failed to perform well in dynamic mobility scenarios.

Recent research has looked at the use of reinforcement learning and optimization for improving computation offloading in highly dynamic vehicular networks. The combination of these two methods greatly increases latency and scalability as well as maintaining adaptability [b20, b21]. Hybrid techniques focus on achieving both fast optimization and intelligent learning in IoV systems. Although there have been significant improvements in this area, several issues still exist, including mobility-based task allocation, communication cost, energy consumption, trust management, and lack of standard benchmarks to evaluate IoV task offloading systems.

Considering the shortcomings witnessed in previous work, it is necessary to design a more intelligent and adaptive task offloading scheme. Therefore, this study suggests a hybrid ACO–DRL based task offloading scheme that integrates ACO and DRL to combine the fast optimization process and intelligent learning capabilities of the two algorithms in order to maximize latency, resource utilization, and performance in IoV networks.

A. Research Gap

Although task offloading in Internet of Vehicles (IoV) systems has improved significantly in recent years, several important challenges still exist. Many existing approaches cannot efficiently handle rapidly changing traffic conditions, unstable network connections, and high vehicle mobility. Edge computing helps reduce delay, but edge servers may become overloaded during heavy traffic, while cloud-based solutions often increase communication latency.

Artificial intelligence-based methods such as Deep Reinforcement Learning (DRL) provide better decision-making, but they require large datasets, longer training time, and higher computational resources. On the other hand, optimization techniques like Ant Colony Optimization (ACO) are faster and less complex but struggle to adapt to continuously changing vehicular environments. In addition, issues related to security, communication overhead, and the lack of standardized evaluation methods still affect the overall reliability of IoV systems. Therefore, there is a need for a more adaptive, scalable, and intelligent hybrid task offloading framework for real-time IoV applications.

III. PROBLEM STATEMENT

The Internet of Vehicles (IoV) is an important technology that enables smart transportation systems through effective communication among vehicles, roadside infrastructure, edge servers, and cloud platforms. However, the rapid increase in connected vehicles creates a high volume of computational tasks that need to be processed quickly and within strict time limits.

Traditional cloud-based task processing methods often experience high communication delays, bandwidth constraints, and network congestion due to the physical distance between vehicles and cloud servers. While edge computing lowers latency by placing computational resources closer to vehicles, issues like dynamic vehicle movement, changing network conditions, uneven resource distribution, and overloaded edge servers still impact system performance.

Current task offloading techniques that rely only on optimization algorithms or machine learning methods may not adequately handle the complexity of the highly dynamic IoV environments. Thus, there is a need for an intelligent and flexible task offloading framework that can minimize task completion time, reduce communication delays, improve resource use, and maintain system stability under varying traffic conditions.

This research tackles these issues by exploring a Hybrid Ant Colony Optimization (ACO) and Deep Reinforcement Learning (DRL) based task offloading framework for effective resource allocation and decision-making in Internet of Vehicles settings.

IV. OBJECTIVES

The main goal of this research is to create and assess an intelligent task offloading framework for Internet of Vehicles (IoV) environments. This framework will use a mix of Ant

Colony Optimization (ACO) and Deep Reinforcement Learning (DRL).

The specific goals of the study are as follows:

- To examine the challenges related to task offloading and resource allocation in dynamic IoV environments.
- To review current task offloading methods based on cloud computing, edge computing, ACO, and DRL techniques.
- To create a Hybrid ACO-DRL framework that can make smart task offloading decisions under changing network conditions.
- To reduce communication delay and overall task completion time through better resource allocation.
- To increase task success rate and system stability in low, medium, and high traffic situations.
- To improve resource use across vehicular, edge, and cloud computing layers.
- To compare the performance of the proposed framework with traditional task processing and existing optimization methods.
- To assess the effectiveness of the proposed model using simulation-based performance metrics such as latency, makespan, throughput, and task success rate.

V. PROPOSED METHODOLOGY

A. System Architecture Overview

The proposed Internet of Vehicles framework uses a mix of edge and cloud computing for fast and smart task handling in changing traffic situations. The architecture has three parts: the Vehicular Layer, Edge Layer and Cloud Layer. The Vehicular Layer has cars with ESP8266/ESP32 NodeMCU. Ultrasonic and infrared sensors. These sensors collect data on obstacle distance, vehicle proximity and traffic conditions. The cars do some data processing themselves to reduce communication and support applications that need fast responses. The Edge Layer acts as a ground for computing between cars and the cloud. A Raspberry Pi 5 works as the edge gateway and local controller. Data are received from cars through the Wi-Fi and NRF24L01 modules. The edge server does computing, temporary storage and resource management. This reduces delay. Improves decision-making. The Cloud Layer provides computing resources for complex tasks that do not need to be done in real-time. It keeps data on vehicles, traffic, and system logs for advanced analysis and optimization. This mix of edge and cloud approaches balances the workload. Uses resources efficiently.

The proposed architecture supports Vehicle-to-Vehicle and Vehicle-to-Infrastructure communication. Wi-Fi is mainly used for car-to-edge communication. NRF24L01 modules allow for short-range communication among cars. This communication framework improves connectivity in changing network conditions. To make task allocation better the system uses a Hybrid ACO-DRL offloading engine. Ant Colony Optimization is used for task allocation. Deep Reinforcement Learning then adjusts offloading decisions based on car movement, resource availability. This hybrid approach speeds up convergence reduces

TABLE I
RESEARCH GAPS IN EXISTING IOV TASK OFFLOADING METHODS

Technique	Strength	Limitation
Edge Computing	Reduces latency by processing tasks near vehicles	Edge servers may become overloaded during heavy traffic conditions.
Cloud Computing	Provides high computational power and storage	Higher communication delay due to distant cloud servers.
ACO-Based Methods	Fast convergence with low computational complexity	Lacks adaptive learning capability in dynamic environments.
DRL-Based Methods	Provides intelligent and adaptive decision-making	Requires large datasets and high training time.
V2V Offloading	Improves resource sharing between nearby vehicles	Security and unstable connectivity remain concerns.
Hybrid Edge-Cloud Models	Balances workload between edge and cloud systems	Communication overhead increases due to coordination.
Blockchain-Based Offloading	Improves trust and data security	Adds extra computational and communication overhead.

communication delay and improves task execution efficiency in Internet of Vehicles environments. The Internet of Vehicles framework benefits, from this edge-cloud architecture. The Internet of Vehicles framework handles tasks efficiently with the help of edge and cloud computing.

B. Step-wise Technical Process Description

1) *Step 1 – Data Acquisition and Local Processing:* Smart vehicles with ESP8266/ESP32 NodeMCU modules collect data in real-time. They use infrared (IR) sensors for this. The data collected includes how away obstacles are, how close other vehicles are and traffic conditions. This data is first. Some simple calculations are done right in the vehicle. This helps to cut down on data being sent and supports applications that need to work quickly like detecting collisions and keeping an eye on safety. The vehicles use this data for things such, as collision detection and safety monitoring.

2) *Step 2 – Vehicular Communication and Edge Reception:* The information that has been processed is sent through Vehicle-to-Infrastructure and Vehicle-to-Vehicle communication systems. Vehicle-to-Infrastructure and Vehicle-to-Vehicle communication systems are very important here. We use Wi-Fi to send data between vehicles and the edge gateway. We also use NRF24L01 modules so that vehicles can talk to each other over distances. The Raspberry Pi 5 edge server gets the requests for tasks puts them in order of importance and checks to see if the network and computational resources are available. It keeps track of the Vehicle-to-Infrastructure and Vehicle-, to-Vehicle communication systems.

3) *Step 3 – Hybrid ACO–DRL Based Task Offloading:* The proposed framework uses Ant Colony Optimization to create a task allocation plan. This plan considers how long it takes for information to travel between nodes, which nodes are available and how much it costs to do computations. Then the Deep Reinforcement Learning part makes the task offloading decisions better. It does this by learning how vehicles move how much bandwidth is available and how resources are used. By combining these two optimization methods the

framework becomes more adaptable. This helps to reduce the time it takes to complete tasks.

Algorithm 1: Intelligent Task Offloading using Hybrid ACO–DRL Approach

Inputs:

- Set of computational tasks: $T = \{t_1, t_2, \dots, t_n\}$
- Vehicle nodes: V
- Edge servers: E
- Cloud servers: C
- Available computational resources
- Current network state (Bandwidth, Communication Delay, Queue Length)

Output:

- Optimized task offloading decision for each computational task

Algorithm Steps:

- 1) Initialize pheromone values τ_{ij} for all communication links.
- 2) Initialize the PPO-based DRL policy network π_θ .
- 3) Collect the current network state:
 - Vehicle mobility
 - CPU availability
 - Available bandwidth
 - Queue length
 - Communication latency
- 4) For each incoming task $t_k \in T$:
 - a) Observe the current system state S_t .
 - b) Generate candidate offloading paths using ACO.
 - c) Select the best edge or cloud node.
 - d) Provide the current state to the PPO-based DRL agent.
 - e) Determine the optimal action A_t (Local, Edge, or Cloud).
 - f) Execute the selected offloading decision.
 - g) Evaluate:
 - Task completion delay

Proposed Hybrid ACO–DRL Based IoV Task Offloading Architecture

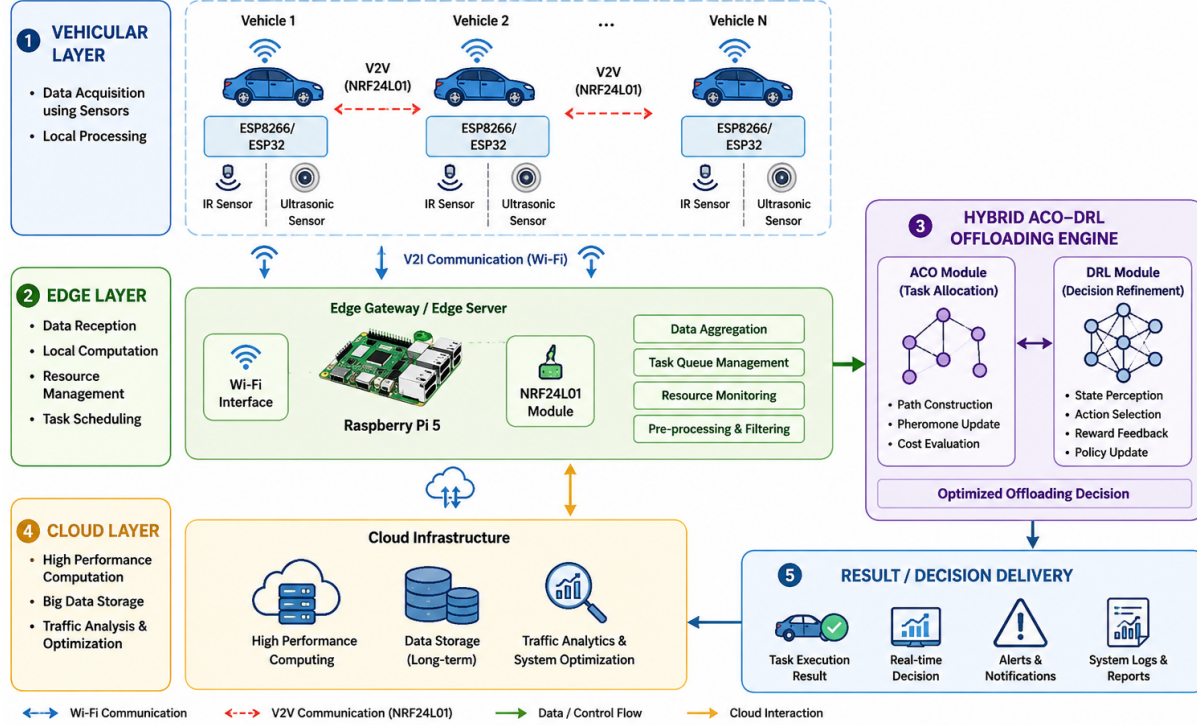


Fig. 1. Proposed Hybrid ACO–DRL based IoV task offloading architecture.

Fig. 1. Proposed Hybrid ACO–DRL based IoV task offloading architecture showing Vehicular Layer, Edge Layer, Cloud Layer, and intelligent task allocation process.

- Resource utilization
- Communication overhead
- Energy consumption

- h) Compute the reward R_t .
- i) Update the PPO policy.
- j) Update the ACO pheromone values.

5) Return the optimized task offloading decisions.

4) *Step 4 – Edge–Cloud Task Execution:* The system works in a way. It looks at what needs to be done. Decides what is the best way to do it. Lightweight tasks that need to be done quickly are done at the edge layer. On the hand tasks that need a lot of computing power are sent to the cloud infrastructure. The cloud infrastructure is really good, at doing tasks. It can do these tasks quickly. Also store a lot of data for a long time. The cloud layer helps to make sure that the system is working well by looking at the traffic and making sure everything is balanced. This means that the cloud layer is supporting the system to work efficiently. The cloud layer is doing this by providing high-performance computation and term data storage and traffic analysis support to the system. The system is using the layer to maintain efficient workload balancing across the system. The cloud layer and the edge layer are working together to make sure that the system is working well.

C. Step 5 - Mathematical Formulation

The proposed Hybrid ACO–DRL framework is mathematically formulated to optimize task offloading decisions in Internet of Vehicles (IoV) environments. The following equations describe the system model.

1) *System State:* The system state observed by the DRL agent at time t is defined as

$$S_t = \{P_v, B_t, C_t, Q_t\} \quad (1)$$

where P_v denotes the vehicle position, B_t represents the available bandwidth, C_t is the available CPU resource, and Q_t denotes the queue length.

2) *Action Space:* The action space is defined as

$$A_t = \{L, E, C\} \quad (2)$$

where L , E , and C represent Local, Edge, and Cloud execution, respectively.

3) *Reward Function:* The reward function is expressed as

$$R = w_1 L + w_2 U - w_3 E \quad (3)$$

where L denotes latency reduction, U represents resource utilization, E is the energy consumption, and w_1 , w_2 , and w_3 are weighting coefficients. The reward formulation follows

the reinforcement learning principle of maximizing long-term cumulative reward while minimizing latency and energy consumption [23], [24].

4) *ACO Transition Probability*: The probability of selecting the next computational node is calculated as

$$P_{ij} = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{k \in N_i} (\tau_{ik})^\alpha (\eta_{ik})^\beta} \quad (4)$$

where τ_{ij} is the pheromone intensity, η_{ij} is the heuristic value, α and β control the influence of pheromone and heuristic information, respectively, and N_i denotes the candidate node set.

5) *ACO Pheromone Update*: The pheromone update rule is defined as

$$\tau_{ij}(t+1) = (1 - \rho)\tau_{ij}(t) + \Delta\tau_{ij} \quad (5)$$

where ρ is the pheromone evaporation coefficient and $\Delta\tau_{ij}$ represents the deposited pheromone.

6) *Communication Delay*: The communication delay is computed as

$$T_{comm} = \frac{D}{B} \quad (6)$$

where D is the task data size and B denotes the available bandwidth.

7) *Computation Delay*: The computation delay is given by

$$T_{comp} = \frac{C_{task}}{f} \quad (7)$$

where C_{task} is the required CPU cycles and f is the processor frequency.

8) *Energy Consumption*: The energy consumption is calculated as

$$E_{total} = P \times T_{comp} \quad (8)$$

where P denotes the processing power.

9) *Overall Task Completion Time*: The total task completion time is expressed as

$$T_{total} = T_{comm} + T_{comp} \quad (9)$$

10) *Optimization Objective*: The objective of the proposed framework is to minimize the execution delay and energy consumption while maximizing resource utilization.

$$\min F = \sum (T_{total} + E_{total}) - U \quad (10)$$

where U represents the overall resource utilization of the edge-cloud infrastructure.

11) Step 6 – Result Delivery and System Optimization:

After the task is done the results are sent back to the layer so that decisions can be made in real time and services can be delivered. The system keeps updating the status of resources how well things are. Information about how things are moving. This helps make decisions, about what tasks to do next. The IoV framework works better because it can respond faster use resources effectively handle more things and is more reliable overall.

D. Complexity Analysis

The computational complexity of the proposed Hybrid ACO-DRL task offloading framework is analyzed in terms of time and space complexity. The ACO module performs path exploration among available computing nodes, while the PPO-based DRL module refines the task offloading decisions based on the observed network state.

1) *Time Complexity*: Let N denote the number of computational tasks, M the number of candidate computing nodes, A the number of ants, and I the maximum number of ACO iterations.

The time complexity of the ACO module is

$$T_{ACO} = O(A \times I \times M) \quad (11)$$

The PPO-based DRL module updates its policy over E training episodes with S state transitions, resulting in

$$T_{DRL} = O(E \times S) \quad (12)$$

Therefore, the overall time complexity of the proposed Hybrid ACO-DRL framework is

$$T_{Hybrid} = O(AIM + ES) \quad (13)$$

where the first term represents the optimization performed by ACO and the second term corresponds to the policy learning process of PPO.

2) *Space Complexity*: The proposed framework stores pheromone values, network state information, and DRL policy parameters. Hence, the overall space complexity is

$$S_{Hybrid} = O(M + S) \quad (14)$$

where M denotes the number of candidate computing nodes and S represents the number of stored network states.

E. Performance evaluation and result

1) *Experimental Setup*: The Hybrid ACO-DRL based task offloading framework was tested using a simulation environment that combined Python and NS-2. We used NS-2 to simulate how vehicles communicate with each other and how the network behaves. On the hand we used Python to implement the ACO and DRL optimization models. The testing environment was set up to mimic an Internet of Vehicles scenario with many moving vehicles, edge servers and cloud resources. We modeled the layer using smart vehicle nodes that have ESP8266 or ESP32 and are equipped with ultrasonic and

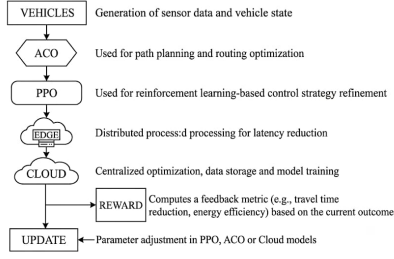


Fig. 2. Workflow of the proposed Hybrid ACO-DRL task offloading framework.

IR sensors. A Raspberry Pi 5 was used as the edge gateway to do some processing and manage tasks. The cloud layer was like a computer server that handles big tasks that need a lot of computing power. The simulation environment took into account real-world factors such as varying traffic density, dynamic mobility patterns changes in bandwidth and different computational resources. The Hybrid ACO-DRL framework was compared to methods, like Local Processing, ACO alone and DRL-based offloading approaches to see how well it works. The Hybrid ACO-DRL framework was evaluated in this environment to see how it performs in Internet of Vehicles conditions.

TABLE II
HYPERPARAMETER SETTINGS FOR HYBRID ACO-DRL

Parameter	Value
PPO Learning Rate	0.0003
Discount Factor (γ)	0.99
Batch Size	64
Training Episodes	500
Number of Ants	30
ACO Iterations	150
Pheromone Importance (α)	1
Heuristic Importance (β)	2
Evaporation Rate (ρ)	0.5
Clip Ratio (ϵ)	0.2

TABLE III
SIMULATION PARAMETERS

Parameter	Value
Simulation Area	1000 m $\times$ 1000 m
Number of Vehicles	20–100
Edge Nodes	1–5
Communication Type	Wi-Fi + NRF24L01
Vehicle Speed	20–80 km/h
Task Size	1–10 MB
Bandwidth	5–20 MHz
Edge Device	Raspberry Pi 5
Simulation Tool	NS-2 + Python
DRL Algorithm	PPO-based DRL
Optimization Method	Hybrid ACO-DRL

2) *Evaluation Protocol*: The proposed framework was tested to see how well it works when vehicles are moving around and network conditions are changing. Many tests were run with both heavy traffic to make sure the results are consistent and reliable. The main goal was to look at how the

TABLE IV
PERFORMANCE UNDER DIFFERENT TRAFFIC CONDITIONS

Traffic Density	Avg. Delay (ms)	Task Success Rate (%)	System Stability
Low Density	120	96	High
Medium Density	145	94	High
High Density	185	89	Moderate

system performs, including things like how long it takes to complete tasks, how well tasks are completed and how much resources are used.

Task completion delay is the time it takes to finish tasks, including the time it takes to communicate and do the actual work. Makespan is the time it takes to finish any task in the system. The task success rate shows what percentage of tasks are completed on time. Communication overhead is the network traffic that is generated when tasks are being sent and resources are being coordinated. Resource utilization shows how well resources are being used across the edge and cloud nodes. System stability shows how well the proposed framework can keep working even when vehicles are moving around traffic is changing and bandwidth is fluctuating.

To see if the proposed approach really works the Hybrid ACO-DRL framework was compared to methods like doing tasks locally using traditional methods- to decide which tasks to offload and using other approaches, to schedule tasks. All of these methods were tested under the conditions so it would be fair to compare them to the Hybrid ACO-DRL framework.

3) *Performance Metrics*:

$$F_k = S_k + t_{comp} + t_{comm} \quad (15)$$

$$\text{Makespan} = \max_{k \in T} (F_k) \quad (16)$$

$$t_{comm} = \frac{D_k}{B_{ij}^{eff}} \quad (17)$$

$$\min M = \min \left(\max_{k \in T} F_k \right) \quad (18)$$

- F_k : Task completion time of task k
 S_k : Start time of task k
 t_{comp} : Computation delay
 t_{comm} : Communication delay
 D_k : Data size of task k
 B_{ij}^{eff} : Effective bandwidth between node i and j
 T : Set of all tasks
 M : Overall makespan (objective function)

4) *Results and Analysis:* The Hybrid ACO-DRL framework does a job of making task offloading more efficient in dynamic IoV environments. When we compare it to using ACO or just DRL, the Hybrid ACO-DRL framework is better because it has lower latency it allocates resources in a better way and it can adapt to changes in mobility. The Hybrid ACO-DRL framework is really good at reducing the time it takes to complete tasks. It does this by balancing the workload between edge and cloud nodes in a way. This means that the Hybrid ACO-DRL framework can complete tasks about 28% faster than the ACO framework. The Hybrid ACO-DRL framework also does a job of reducing makespan when there are a lot of vehicles. The DRL part of the framework is really good at learning and adjusting to changes, which helps make sure that tasks are allocated in the way possible. The ACO part of the framework is really good at finding the solution quickly. When it comes to completing tasks the Hybrid ACO-DRL framework is very good. It can complete about 94% of tasks. The Hybrid ACO-DRL framework also reduces the amount of communication overhead. It does this by preprocessing data at the edge of the network and managing bandwidth in a way. This means that the framework can keep working even when the vehicles are moving really fast and the network is congested. The Hybrid ACO-DRL framework is also very good at balancing the workload between the edge and the cloud. This helps make sure that the edge servers do not get overloaded, even when there is a lot of traffic. The Hybrid ACO-DRL framework is really good, at using resources in a efficient way.

5) *DRL Performance Analysis:* Fig. 3 shows the final performance matrix obtained from the DRL model. The results indicate stable learning convergence and efficient decision-making under varying traffic conditions.

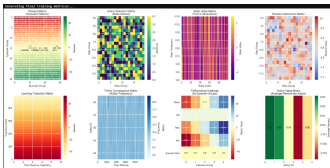


Fig. 3. DRL Final Performance Matrix showing the learning convergence and decision-making effectiveness of the Deep Reinforcement Learning agent under varying network conditions.

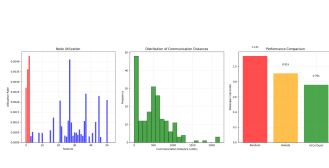


Fig. 4. ACO Communication Distribution Graph illustrating task allocation and communication load distribution among vehicular, edge, and cloud nodes.

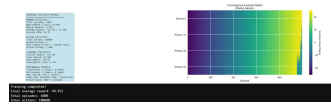


Fig. 5. ACO Optimized Matrix representing the optimized task allocation strategy generated through Ant Colony Optimization.



Fig. 6. Network Graph depicting communication links among vehicles, edge servers, and cloud infrastructure in the proposed IoV framework.

6) *Comparative Analysis:* The Hybrid ACO-DRL approach works well because it combines the best of both Ant Colony Optimization (ACO) and Deep Reinforcement Learning (DRL) methods. The ACO part helps find good solutions fast by checking out lots of possibilities and using clues to find good paths. This helps get close to the solution quickly. On the hand the DRL part helps make good decisions by learning from what happens in the environment like changes in the network task loads and available resources. When you put ACO and DRL together the Hybrid ACO-DRL approach gets better at finding solutions and using what it already knows. This helps in getting tasks done faster succeeding often and using resources better in changing vehicle networks. The Hybrid ACO-DRL approach is also better at handling things at once and working well even when things change quickly and unexpectedly. So it is a fit, for big vehicle networks where things are always changing. The Hybrid ACO-DRL approach makes use of ACO and DRL to achieve this. The ACO and DRL are used together in Hybrid ACO-DRL. The Hybrid ACO-DRL is good because of ACO and DRL.

TABLE V
PERFORMANCE COMPARISON OF OFFLOADING METHODS

Method	Average Delay	Task Success Rate	Resource Utilization	Scalability
Local Execution	High	Low	Poor	Limited
ACO-Based Offloading	Medium	Moderate	Moderate	Good
DRL-Based Offloading	Low	High	Good	Moderate
Hybrid ACO-DRL	Very Low	Very High	Excellent	Excellent

VI. CONCLUSION

This study addressed the challenge of efficient task offloading in the Internet of Vehicles (IoV), where high vehicle mobility, dynamic network conditions, and limited computational resources significantly impact system performance. A hybrid edge-cloud offloading framework was investigated to optimize task allocation while balancing latency, resource utilization, and service reliability.

The experimental analysis demonstrated that the proposed approach effectively reduces task execution delay, improves task completion rates, and enhances overall system stability under varying traffic conditions. By incorporating mobility-aware decision-making and distributed computing capabilities, the framework adapts efficiently to changing vehicular environments and supports real-time application requirements.

The results highlight the importance of intelligent offloading strategies in overcoming the limitations of traditional cloud-centric architectures. The integration of optimization techniques and adaptive decision-making mechanisms enables more efficient utilization of edge resources while maintaining reliable communication among connected vehicles.

Overall, the proposed framework provides a practical and scalable solution for next-generation IoV systems. As connected and autonomous vehicles become increasingly prevalent, intelligent task offloading will play a critical role in ensuring low-latency communication, efficient resource management, and improved quality of service for future smart transportation networks.

VII. FUTURE SCOPE

The rapid evolution of the Internet of Vehicles (IoV) is expected to increase the demand for intelligent, adaptive, and autonomous task offloading mechanisms. Although the proposed hybrid ACO-DRL framework demonstrates promising performance in optimizing task allocation and reducing latency, several opportunities exist for further enhancement.

First, advanced security mechanisms such as blockchain technology, federated learning, and zero-trust architectures can be integrated into the framework to ensure secure and privacy-preserving communication among vehicles, edge nodes, and cloud servers. These technologies can strengthen trust management while protecting sensitive vehicular data during task offloading operations.

Second, the proposed model can be extended to leverage next-generation communication technologies, including 5G and upcoming 6G networks. The ultra-low latency, high reliability, and enhanced bandwidth offered by these networks can significantly improve offloading decisions for time-critical applications such as autonomous driving and real-time traffic management.

Another important direction involves real-world deployment and field testing. While simulation-based evaluation provides valuable insights, implementing the framework in practical vehicular environments would help validate its effectiveness under real traffic conditions, communication uncertainties, hardware limitations, and signal interference scenarios.

Future research may also focus on energy-aware optimization techniques. Developing battery-efficient decision-making strategies can be particularly beneficial for electric and hybrid vehicles, where energy conservation is a critical concern. Incorporating energy consumption metrics into the optimization process can further enhance the sustainability of IoV systems.

In addition, the scalability of the framework can be improved to support large-scale smart city infrastructures. By

enabling efficient coordination among thousands of connected vehicles, roadside units, and edge servers, the proposed approach can contribute to intelligent traffic management, cooperative perception, and enhanced urban mobility services.

Overall, these future enhancements have the potential to create a more secure, scalable, energy-efficient, and intelligent IoV ecosystem capable of supporting fully autonomous transportation and next-generation vehicular communication networks.

REFERENCES

- [1] M. Chen, W. Saad, and C. Yin, "Echo-Liquid State Deep Learning for IoV Task Offloading," *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 3549–3562, 2020.
- [2] X. Hou, Y. Ren, S. Yu, and W. Cheng, "Vehicular Edge Computing for Safety Applications: Architecture and Performance," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 3, pp. 2492–2506, Mar. 2020.
- [3] K. Zhang, Y. Mao, S. Leng, A. Vinel, and Y. Zhang, "Delay and Energy Efficient Task Offloading for IoV Using MEC," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 7, pp. 4812–4822, 2021.
- [4] C. Liu, J. Guo, X. Wang, and J. Tang, "Deep Reinforcement Learning for Vehicular Task Offloading in Dynamic IoV Systems," *IEEE Transactions on Mobile Computing*, vol. 21, no. 6, pp. 2030–2044, Jun. 2022.
- [5] M. Sookhak, H. Talebian, N. Abbasi, and N. Zhang, "Mobility-Aware Edge Computing in IoV," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2571–2601, 2021.
- [6] Z. Ning, P. Dong, X. Wang, and J. Rodrigues, "AI-Driven Edge Computing for Resource Management in IoV," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 7, pp. 4232–4244, 2021.
- [7] J. Sun, Z. Jiang, and Q. Pei, "MEC-Based Low-Latency Scheduling for IoV," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 5693–5705, 2020.
- [8] M. Alghamdi and X. Liao, "Hybrid Edge-Cloud Architecture for IoT/IoV Task Processing," *IEEE Access*, vol. 8, pp. 144536–144548, 2020.
- [9] P. Zhang and L. Chen, "Peer-to-Peer Task Offloading in V2V Networks," *IEEE Internet of Things Journal*, vol. 10, no. 2, pp. 1674–1686, 2023.
- [10] P. Tang, X. Li, and Y. Zhou, "Game-Theoretic Offloading for Vehicular Networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 9, pp. 5670–5683, 2021.
- [11] S. Li, Q. Wu, and K. Zhang, "Blockchain-Enabled Secure and Fair Task Offloading in IoV," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 12, pp. 24126–24137, 2022.
- [12] J. Du, C. Jiang, and K. Han, "Mobility Prediction and Offloading in Vehicular Edge Networks," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 4, pp. 3398–3412, 2021.
- [13] T. Wang, Y. Zhao, and Y. Guo, "Energy-Aware Task Allocation for Electric Vehicles in Edge Computing," *IEEE Transactions on Green Communications and Networking*, vol. 6, no. 3, pp. 1280–1292, 2022.
- [14] H. Zhang, M. Xiao, G. Wu, and S. Han, "Scalable Offloading Metrics in IoV Systems," *IEEE Transactions on Mobile Computing*, vol. 20, no. 4, pp. 1450–1464, 2021.
- [15] A. Ghaboosi, M. Bagheri, and R. Tafazolli, "Bandwidth-Aware Scheduling for IoV MEC Systems," *IEEE Transactions on Communications*, vol. 70, no. 11, pp. 7320–7333, 2022.
- [16] L. Gao, X. Huang, and H. Li, "Mobility-Aware Decision Engines for Real-Time Offloading," *IEEE Transactions on Wireless Communications*, vol. 20, no. 10, pp. 6342–6356, 2021.
- [17] Q. Liu, Z. Gao, and Y. Sun, "Hybrid Offloading Architectures for Next-Generation IoV," *Future Generation Computer Systems*, vol. 124, pp. 129–141, 2021.
- [18] R. Yu, J. Ding, and F. Cao, "Real-Time Secure Offloading in Fast Vehicular Networks," *Computer Networks*, vol. 204, pp. 108–122, 2022.
- [19] J. Zhao, X. Cheng, and L. Yang, "Limitations of Simulation-Based IoV Studies: Real-World Constraints and Challenges," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–34, 2023.
- [20] Y. Wang, Z. Ning, X. Wang, and L. Guo, "Deep Reinforcement Learning for Computation Offloading in Mobile Edge Computing: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 1, pp. 1–28, 2024.

- [21] Z. Ning, P. Dong, X. Wang, J. Rodrigues, and F. Xia, "Artificial Intelligence for Vehicular Edge Computing: Recent Advances and Future Directions," *IEEE Internet of Things Journal*, 2024.
- [22] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A Survey on Mobile Edge Computing: The Communication Perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [23] S. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.